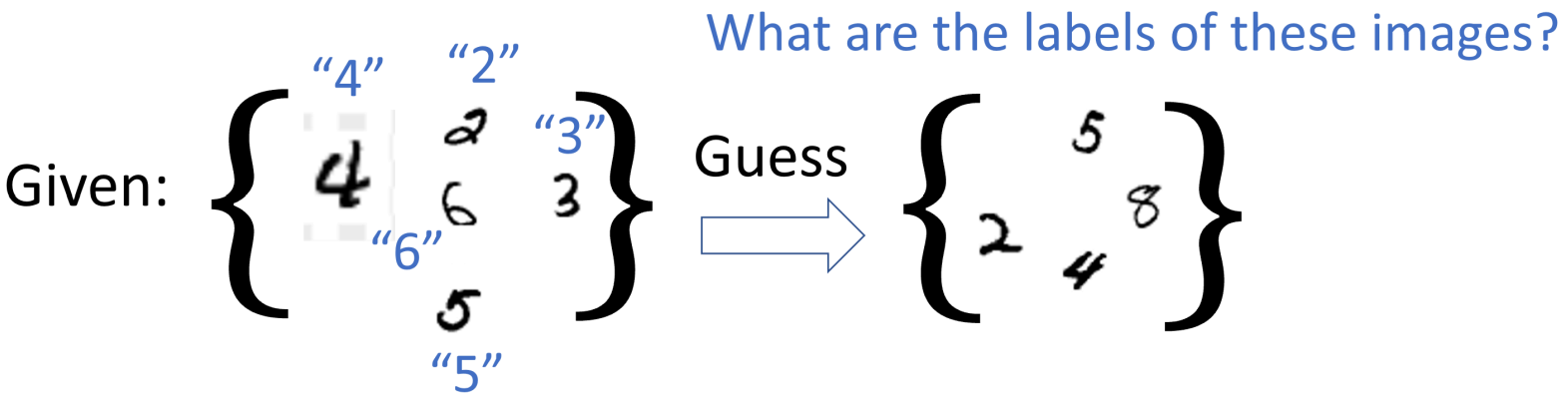
Assessed Coursework 1

Task: Given a data set (MNIST) containing images of handwritten digits, implement a image recognition algorithm (perceptron).

Prediction Task

- In this CW, the goal is to use images and labels in one dataset to predict labels of images in another dataset, where labels are NOT observed.



Images as Vectors

- As we mentioned in the previous tutorial, images are stored as flattened matrices (in row/col major order) in vectors.
- Each image is stored as a vector in this coursework.

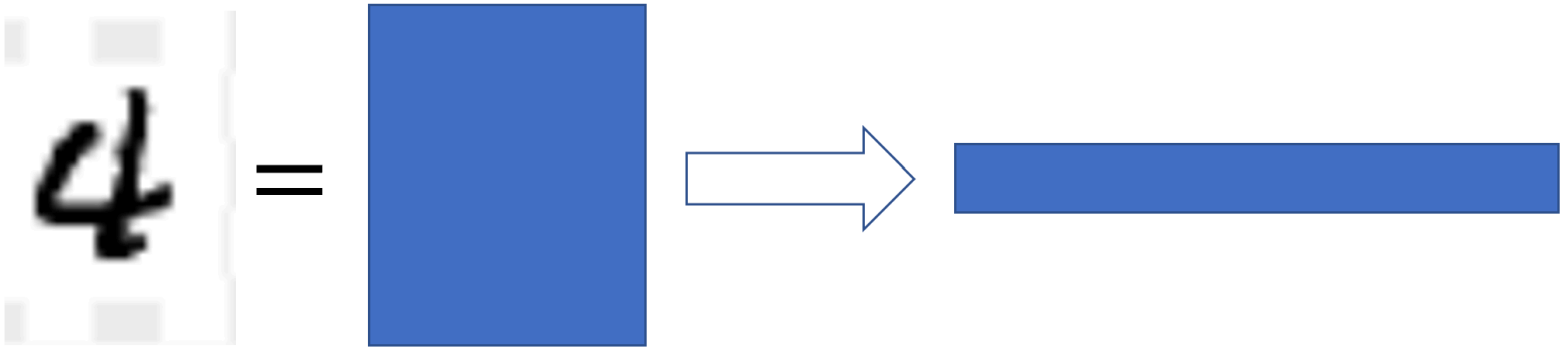
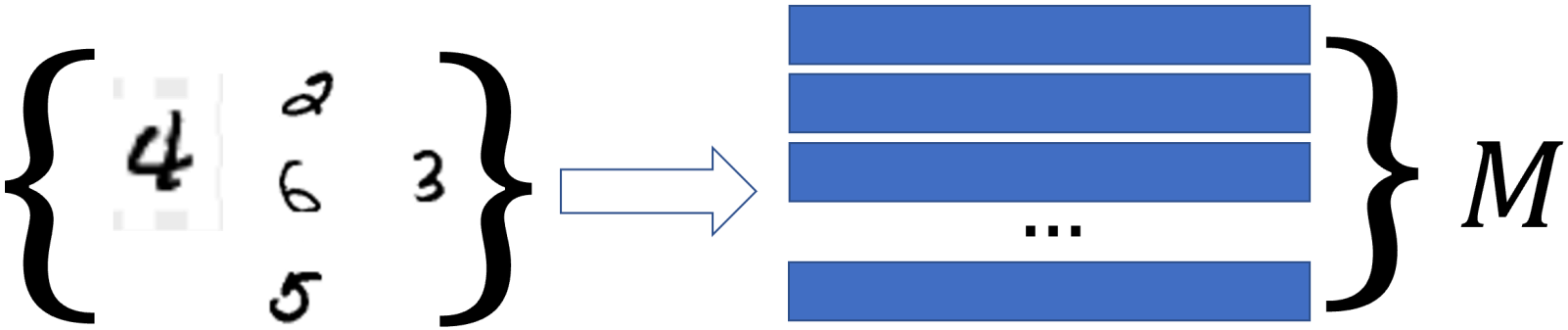


Image Sets

- In this CW, we are dealing with sets of images. Stacking all the image vectors together, you get a matrix.

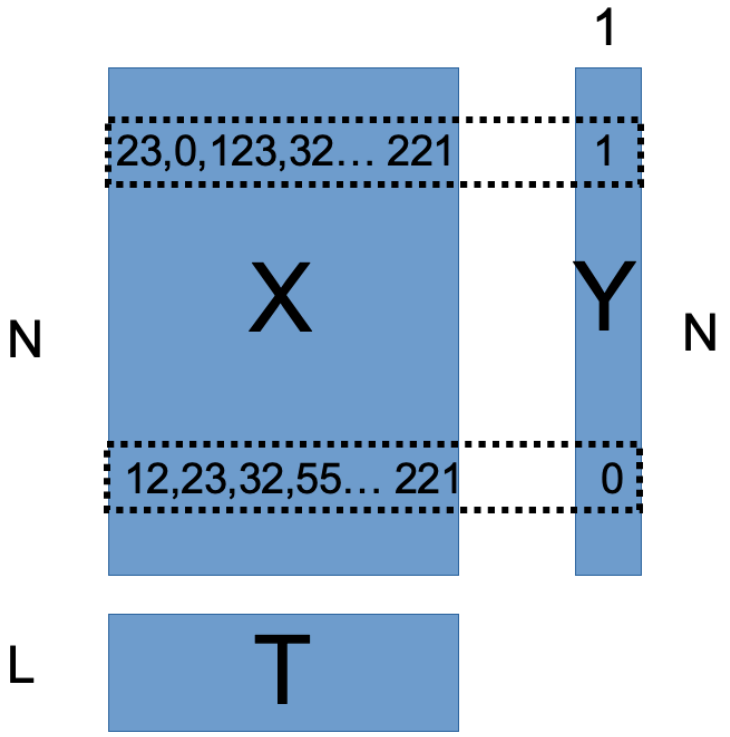


- Specifically, there are two sets of images in this CW, and they are represented by two matrices X, T.

Part I, The Data Set

- CW folder contains 3 `.matrix` files storing 3 matrices.
  - `X.matrix` stores an N by M^2 matrix X where each row is a grayscale M by M image stored in row-major order.
  - `T.matrix` stores an L by M^2 matrix T where each row is an M by M **test image** in row major order.
  - `Y.matrix` stores an N by 1 matrix Y where each row is a scalar, indicating the label of the corresponding row in X.
  - X and Y together are called "training set" in machine learning, while T is the "testing set". Y is called the "labels" of X.

Part I, Data Structure



- If $Y_i = 1$, then the image X_i is a handwritten digit 1. If $Y_i \neq 1$, the image X_i is NOT a handwritten digit 1.

Part I, Load Dataset and Observe

- `matrix` struct has been defined for you.

```
struct matrix
{
    int numrow;
    int numcol;
    int *elements;
};
```

- The code for loading these matrices from files have been provided to you.
- By simply running the skeleton code, you should see some basic statistics of X, Y and T .
    - What are M, N and L?
    - How many images in the training set X are digit 1?

Part II Helper Functions

Before your `main` function, Write a few helper functions:

```
int get_elem(Matrix M, int i, int j)
```

- returns the `i, j` th element of matrix `M`. In this coursework, `i, j` are **zero-based indices**.

```
void set_elem(Matrix M, int i, int j, int value)
```

- assign `value` to the `i, j` th element of matrix `M`

```
int dot(Matrix A, Matrix B)
```

- return the dot product between to column vectors A and B, stored in `Matrix` structs.

Part III Make New Labels

- To make things simpler, let us convert Y into a new matrix:
- In `main` function, create a new matrix $\tilde{Y}$ with the same dimension as Y. For all i, j,
  - If $Y_{i,j} = 1$, then $\tilde{Y}_{i,j} = 1$.
  - If $Y_{i,j} \neq 1$, then $\tilde{Y}_{i,j} = -1$.

Part III Create a Weight Vector

- In `main`, create a "Weight Vector", $\beta \in \mathbb{Z}^{(M^2+1)\times 1}$.
- The weight vector is an integer vector.
- The number of rows of β is the same as the number of columns of X **plus one**.
- β shall be initialized with zeros.
- For both β and $\tilde{Y}$, use heap allocation.

Part III Training

Now, we will update the weight vector β, which will be instrumental to our classification algorithm later.

- Write `void update(Matrix u, Matrix beta, int y)`, where u and β are both column vectors with the same size.
- The function performs the following operations:
    i. If $y \cdot \langle u, \beta \rangle > 0$, do nothing.
    ii. If $y \cdot \langle u, \beta \rangle \leq 0$, update β by adding $y \cdot u$ to β.
    - Here, $y \cdot u$ is a scalar-vector product.
- The elements in β is modified after calling the `update` function.

Part III Training

Back in `main`, write c code to reflect the following pseudo code:

- Repeat the operation below 100 times
  - For each row x_i in X
    - create a new column vector $\tilde{x}_i$, where the first element is 1, and the rest are the elements in x_i.
    - `update` ($\tilde{x}_i$, β, $\tilde{Y}_{i,0}$)

Part IV Guessing Labels

Still in `main`, using the updated weight parameter β, we guess the labels of the test images in T. Write c code to reflect the following pseudo code:

- For each row t_i in T:
  - create a new column vector $\tilde{t}_i$, where the first element is 1, and the rest are the elements in t_i.
  - If $\langle \tilde{t}_i, \beta \rangle > 0$, print `1`.
  - If $\langle \tilde{t}_i, \beta \rangle \leq 0$, print `not 1`.
  - After your guess, you can optionally print the image stored in T_i to the console to validate your guess.

CW2: Marking Criteria

- Submitting correct code (10%)
    - Submitting a C file with **the correct name**.
    - Your code compiles and runs **without major error** such as **crash**, **infinite loop**.
        - It will be tested using `gcc` in the lab pack.
- Part II 20%
- Part III 30%
- Part IV 20%
- Good Coding Practice (20%)
    - Good code format

CW2: Dos and Don'ts

- You can discuss with your classmates about general strategies but write your own code!
- Don't give your code to other students.
- Review relevant previous lab sessions before you start.
- You need to add a reference in the comments. Copy and pasting code from internet (including chatGPT) without citation is not allowed.
- You are only allowed to use standard features of C.
    - You can use `stdio.h`, `stdlib.h`, `limits.h` and `math.h`.
    - If you want to use other libraries, consult with the lecturer or TA beforehand.